



Réunion 5

Clément Sinon

Table des matières

1	Itération 1	3
1.1	Feedback sur la première semaine de programmation	3
1.2	Changement des binômes	3
2	Organisation	4
2.1	Mise à jour	4
2.2	UML	4
2.3	Burndown charts	4
3	Programmation	5
3.1	Refactoring	5
3.2	Conditions pour les commits	5
3.3	Utilisation des constantes	5

Chapitre 1

Itération 1

1.1 Feedback sur la première semaine de programmation

Un tour de table sera fait pour que chacun exprime son ressenti et donne son retour sur la première semaine de l'itération 1.

1.2 Changement des binômes

Comme à chaque semaine, il nous faut réassigner les binômes pour le *pair programming*. Cette assignation, à partir de la deuxième semaine, se ferait à priori aléatoirement.

Tâche 1.1

Réassigner les binômes.

Chapitre 2

Organisation

2.1 Mise à jour

Il a été observé qu'il est difficile de garder trace du travail qui a été accompli par les autres. Il est aussi difficile de comprendre certains choix des autres membres.

Une approche différente à la façon de mettre les autres au courant de son avancée pourrait être recherchée. Il est proposé de réduire le nombre de postes faits sur le serveur Discord à un par histoire. Une approche avancée par le cours est de faire des *stand-up meetings* réguliers.

Vote 2.1

Comment partager son avancée ?

2.2 UML

Des diagrammes UML pourraient aider les membres à mieux s'orienter dans le code et pourraient être utiles pour mieux identifier les problèmes de *design* dans le programme. Ils seraient aussi appréciés par les assistants et les professeurs lorsqu'ils analysent notre projet. Il est probable qu'on doive en faire pour la présentation orale en fin de quadrimestre.

Un ou plusieurs membres pourraient être assignés à la maintenance de diagrammes UML.

Tâche 2.1

Maintenir des diagrammes UML.

2.3 Burndown charts

Nous sommes sensés maintenir une *burndown chart* pour calculer notre vélocité. Cette tâche pourrait être déléguée à quelqu'un.

Tâche 2.2

Maintenir une *burndown chart*.

Chapitre 3

Programmation

3.1 Refactoring

Selon le principe de *collective ownership* d'XP, chacun est encouragé à *refactor* l'entièreté de la *codebase* à tout moment. Cependant, trop de *refactoring* peut ajouter trop de chaos dans le processus de programmation. D'autant plus si des petits changements sont faits dans un sens puis dans l'autre. Une proposition est de dédier un temps dans la semaine pour retoucher le code ensemble.

Vote 3.1

Comment aborder le *refactoring* ?

3.2 Conditions pour les commits

Du code buggé ou qui ne valide pas les tests est parfois soumis sur le dépôt GitLab. Cela peut nuire à la productivité des autres membres.

Il est proposé d'établir comme règle de ne soumettre que du code valide.

Vote 3.2

Soumission de code invalide : autorisée ou non ?

3.3 Utilisation des constantes

Une proposition est de centraliser les constantes du code dans une classe dédiée. Cela permettrait de réduire la redondance et de rendre les constantes plus faciles d'accès.

Vote 3.3

Centraliser les constantes du programme ?